

C++ Tests

TLDR

Think of each test as normal C++ code that gets wrapped in two layers:

- GoogleTest checks the actual behavior. You write these checks in `*Test.cpp` files with `TEST(SuiteName, CaseName)`.
- CTest runs those GoogleTest cases from the build tree. CI uses CTest too.

To add tests, usually do three things:

1. Put the test code in the right `tests/cpp/<library>/<area>` directory.
2. Register the test executable in that area's `CMakeLists.txt` with `add_moos_cpp_test`.
3. Add useful labels so people can run just the library, component, or app-impact area they care about.

Most local runs look like this:

```
ctest --test-dir build -L geometry --output-on-failure
```

When editing one file, run the executable directly with a GoogleTest filter:

```
./bin/test_geometry_arcutils --gtest_filter='ArcUtilsIntersectionTest.*'
```

Do not launch a MOOS mission for these tests unless the behavior truly depends on live MOOS process interaction.

This tree holds MOOS-IvP C++ tests that run as normal unit/integration-style executables. They do not launch missions, MOOSDBs, or pShare ports. They are built by CMake, registered in CTest, and written with GoogleTest.

The suite is designed for targeted regression work. A developer should be able to run:

- all C++ tests with `ctest --test-dir build --output-on-failure`
- one library with a label, such as `ctest --test-dir build -L geometry`
- one component with a label, such as `ctest --test-dir build -L XYArc`
- one GoogleTest suite with an executable filter, such as `./bin/test_geometry_arcutils --gtest_filter='ArcUtilsIntersectionTest.*'`

Mental Model

Each test target has three layers:

- CMake target: one executable, created with `add_moos_cpp_test`
- GoogleTest suites: source-level groups such as `XYPointTest` or `ZAICPeakTest`

- CTest cases: one registered test per GoogleTest test case, with labels

CTest is the main runner because it understands labels and is what CI uses. GoogleTest is the assertion framework inside each executable.

CTest And GoogleTest

GoogleTest is where tests are written. A test source usually contains cases like this:

```
TEST(XYPointTest, ParsesViewerPointWithLabelAndColor)
{
    XYPoint point = string2Point("x=10,y=20,label=alpha,vertex_color=red");

    EXPECT_TRUE(point.valid());
    EXPECT_DOUBLE_EQ(point.x(), 10);
    EXPECT_DOUBLE_EQ(point.y(), 20);
    EXPECT_EQ(point.get_label(), "alpha");
}
```

The first argument, `XYPointTest`, is the GoogleTest suite. The second argument is the case name. Together they form the GoogleTest filter name:

```
XYPointTest.ParsesViewerPointWithLabelAndColor
```

CTest is the runner around those GoogleTest cases. CMake discovers each GoogleTest case and registers it as a separate CTest case with the executable name prepended:

```
test_geometry_xyformatutils_point.XYPointTest.ParsesViewerPointWithLabelAndColor
```

That means:

- use CTest when you want CI-equivalent behavior, labels, or broad subsets
- use `--gtest_filter` on the executable when iterating on one suite or case
- use CTest labels when the boundary is a library, component, app impact, or cross-cutting domain

Run Tests

Build and run everything:

```
./build.sh
ctest --test-dir build --output-on-failure
```

If the build cache points at the wrong MOOS-IvP checkout, reconfigure once:

```
cmake -S . -B build -DMOOSIVP_SOURCE_TREE_BASE=/path/to/moos-ivp
cmake --build build
ctest --test-dir build --output-on-failure
```

Run a focused CTest label:

```
ctest --test-dir build -L geometry --output-on-failure
ctest --test-dir build -L XYArc --output-on-failure
ctest --test-dir build -L BHV_Waypoint --output-on-failure
ctest --test-dir build -L pMarineViewer --output-on-failure
```

CTest label matching is regex-based. Anchor broad labels when needed:

```
ctest --test-dir build -L '^behaviors$' --output-on-failure
```

Run one executable directly:

```
./bin/test_geometry_arcutils
./bin/test_ivpbuild_zaic
```

Run one GoogleTest suite or case directly:

```
./bin/test_geometry_arcutils --gtest_filter='ArcUtilsIntersectionTest.*'
./bin/test_ivpbuild_zaic --
gtest_filter='ZAICPeakTest.CoursePeakWrapsAcrossNorthForHelmHeadingDomains'
```

List labels and discovered tests:

```
ctest --test-dir build --print-labels
ctest --test-dir build -N
```

Layout

Use this structure:

```
tests/cpp/<library>/CMakeLists.txt
tests/cpp/<library>/<area>/CMakeLists.txt
```

```
tests/cpp/<library>/<area>/*Test.cpp
tests/cpp/support/*
```

Examples:

```
tests/cpp/geometry/XYFormatUtilsPoint/XYFormatUtilsPointTest.cpp
tests/cpp/ivpbuild/ZAIC/ZAICTest.cpp
tests/cpp/mbutil/CoreString/MBUtilsTest.cpp
```

The top-level library directory should match the upstream library or practical repo bucket. The `<area>` directory should group a coherent component family, not a random collection of unrelated tests.

Shared helpers live in `tests/cpp/support` and are automatically included for every target created with `add_moos_cpp_test`.

CMake Pattern

Every leaf test target should use `add_moos_cpp_test`. It is this repo's CMake wrapper for a C++ test executable. It creates the executable, links GoogleTest, adds shared test includes, asks CMake to discover the GoogleTest cases, and attaches CTest labels/environment settings.

Use it instead of hand-writing `add_executable`, `target_link_libraries`, and `gtest_discover_tests` for normal C++ tests:

```
add_moos_cpp_test(test_geometry_xyformatutils_point
  SOURCES
    XYFormatUtilsPointTest.cpp
  LIBS
    geometry
    mbutil
  LABELS
    geometry
    formats
  SUITE_LABELS
    XYPointTest=XYPoint
    XYFormatUtilsPointTest=XYFormatUtilsPoint
)
```

Supported arguments:

- **SOURCES**: test files and any extra app/library source files
- **LIBS**: libraries to link
- **INCLUDES**: target-specific include directories
- **DEFINES**: target-specific compile definitions
- **LABELS**: CTest labels applied to every discovered case in the executable
- **SUITE_LABELS**: labels applied only to matching GoogleTest suites, written as `SuiteName=LabelA,LabelB`

- **WORKING_DIRECTORY**: optional CTest working directory
- **DISCOVERY_MODE**: optional GoogleTest discovery mode, such as **PRE_TEST**
- **ENVIRONMENT**: optional CTest environment entries such as **NAME=value**

Keep target setup inside **add_moos_cpp_test** when possible. Avoid follow-up **target_link_libraries**, **target_include_directories**, or **set_tests_properties** blocks unless there is a clear local reason.

For external MOOS-IvP dependencies, use CMake discovery. Prefer imported targets or variables from **find_package**; otherwise use **find_library** and **find_path**. Do not link directly to hardcoded **.dylib**, **.so**, or platform build-output paths from test **CMakeLists.txt** files.

Labels

Labels are a public interface for developers and CI. Treat them as carefully as test names.

Use target-wide **LABELS** only when the label truthfully applies to every CTest case in the executable. Common target-wide labels are:

- library labels: **geometry**, **mbutil**, **ivpbuild**, **bhvutil**
- domain labels: **math**, **formats**, **grid**, **populators**, **viewer_plugins**
- app-impact labels: **pHelmIvP**, **pMarineViewer**, **pMarinePIDV22**

Use **SUITE_LABELS** for concrete component labels in mixed executables:

```
LABELS
  geometry
  formats
SUITE_LABELS
  XYRangePulseTest=XYRangePulse
  XYFormatUtilsRangePulseTest=XYFormatUtilsRangePulse
  XYCommsPulseTest=XYCommsPulse
  XYFormatUtilsCommsPulseTest=XYFormatUtilsCommsPulse
```

Rules:

- every CTest case must have at least one useful label
- no duplicate labels on a case
- concrete class/tool labels should not spill across unrelated suites
- family labels may be broad when the whole executable is the family
- app-impact labels mean the test protects behavior used by that app; they do not mean the app itself is launched
- prefer canonical MOOS-IvP names, including existing capitalization such as **XYPolygon**, **ZAIC_PEAK**, **BHV_Waypoint**, or **VIEW_POLYGON**

Test Quality Standard

The goal is not a smoke test per class. Each suite should be deep enough that a changed MOOS-IvP checkout produces a specific, actionable failure.

For parsers and format utilities, cover:

- canonical MOOS-IvP payloads, such as `VIEW_POINT`, `VIEW_POLYGON`, `VIEW_SEGLIST`, `VIEW_GRID`, `NODE_REPORT`, and behavior config strings
- abbreviated and standard forms when both are supported
- labels, colors, render hints, source/type metadata, duration, active flags, and other viewer-facing fields
- malformed input, empty input, duplicate fields, unknown fields, numeric quirks, and tolerated legacy behavior
- serialization, spec output, or round-trip behavior when stable

For geometry and math tools, cover:

- nominal cases plus boundaries
- degenerate inputs: empty objects, one-point objects, zero radius, zero length, zero speed, zero duration, invalid domains
- wraparound headings and angle intervals across north
- crossing and non-crossing cases
- distance/value outputs as well as boolean classifiers
- ownership/copy/cache behavior when objects maintain internal state
- MOOS-IvP use cases such as waypoint lanes, loiter regions, OpRegion polygons, obstacle polygons, turn arcs, contact CPA, Dubins lookahead, search grids, viewer shapes, and IvP course/speed domains

For app-facing helpers, cover the payloads and state transitions used by the app, but keep the test local unless the target is a mission harness.

Naming

Use suite names that identify the unit under test:

```
TEST(XYConvexGridTest, ParsesViewGridSpecWithCellUpdatesAndLimits)
TEST(ZAICPeakTest, CoursePeakWrapsAcrossNorthForHelmHeadingDomains)
```

Good test case names state the condition and expected behavior. Prefer:

```
Tool0rObjectTest.ContextAndExpectedBehavior
```

Avoid names like `Basic`, `Works`, `Test1`, or names that only restate the API call. A failure should tell the reader what contract was broken.

Assertions

Use direct, narrow assertions. Prefer:

- exact string checks for stable serialized specs

- tolerance checks for floating-point geometry/math values
- explicit false-path checks for malformed input
- separate assertions for independent behavior when a failure needs a precise location

When testing a known upstream quirk, name it clearly in the test case. These tests intentionally protect current behavior until the project decides to change it.

MOOS-IvP Grounding

Tests should include abstract edge cases when they improve coverage, but each library family should also include examples grounded in actual MOOS-IvP usage.

Good grounded inputs include:

- viewer postings: `VIEW_POINT`, `VIEW_POLYGON`, `VIEW_SEGLIST`, `VIEW_GRID`, `VIEW_RANGE_PULSE`, `VIEW_COMMS_PULSE`
- contact reports: `NODE_REPORT`, `CONTACT_INFO`, CPA-related geometry
- behavior configs: waypoint, loiter, op-region, obstacle, cut-range, shadow, and avoid-collision style parameters
- IvP domains and functions: course/speed domains, ZAICs, AOFs, PDMaps, IPF strings, and function encoder packets
- log/app utilities: `.alog`-style lines, helm summaries, appcast/realmcaster strings, command posts, and mission-eval conditions

Do not require a launched mission just to test a pure library contract. Use a mission harness only when the behavior depends on live MOOS process interaction.

Adding Coverage

Before adding a new test family:

1. Find the upstream library or app source under `moos-ivp/ivp/src`.
2. Check nearby tests for the closest pattern.
3. Choose the smallest coherent `<area>` bucket.
4. Add or extend a leaf `CMakeLists.txt` with precise `LABELS` and `SUITE_LABELS`.
5. Add focused GoogleTest suites with grounded and edge-case examples.
6. Run the focused label first, then the invariant checker.

Useful local commands:

```
ctest --test-dir build -L XYFormatUtilsPoint --output-on-failure
ctest --test-dir build -L geometry --output-on-failure
python3 scripts/check_cpp_tests.py \
  --build-dir build \
  --moos-src /path/to/moos-ivp/ivp/src
```

Run the full suite before handing off broad changes:

```
ctest --test-dir build --output-on-failure
```

Invariants

`scripts/check_cpp_tests.py` enforces repo-level C++ test invariants. It checks registration, labels, source-file references, stale discovery placeholders, platform-specific link literals, and optional upstream `lib_*` to `tests/cpp` bucket mapping.

The GitHub Actions workflow uses the same CTest registry and checker. Its `cpp_test_filter` dispatch input accepts:

- `all`: run the full C++ suite
- any CTest label: run a focused subset, such as `geometry`, `BHV_Waypoint`, `LogPlot`, or `pMarineViewer`

If a new convention cannot be expressed in this README and enforced or audited reasonably, reconsider the convention.